

Séquence 3 – Parsing, structuration et traçabilité

De la ligne brute à la décision contestable

École de l'air et de l'espace

Une décision contestée doit remonter à sa source

“35,4 °C dans le shelter batteries : inspection déclenchée.”

Question décisionnelle

Quel message précis, quelle transformation exacte et quelles limites assumées permettent de justifier cette décision si on vous la conteste plus tard ?

À la fin, vous saurez décider si l'indicateur est défendable

- ▶ parser des messages JSON sans jamais corriger silencieusement une valeur suspecte ;
- ▶ structurer le résultat sous un schéma explicite, documenté champ par champ ;
- ▶ relier chaque ligne de la table structurée au message brut exact dont elle provient ;
- ▶ distinguer clairement intégrité (les octets n'ont pas changé), authenticité (l'auteur est bien celui qu'il prétend) et vérité physique (la mesure reflète la réalité) ;
- ▶ traiter un quasi-doublon comme un signal à documenter, jamais comme une ligne à supprimer sans règle ;
- ▶ qualifier séparément une confiance technique (la chaîne est-elle auditée ?) et une confiance opérationnelle (peut-on agir dessus ?).

Quatre productions rythment les 240 minutes

1. TP 1 : produire `batch001_structured.csv` à partir des 15 messages bruts ;
2. TP 2 : défendre un schéma commenté, champ par champ ;
3. TP 3 : rejouer manuellement le lien entre une ligne de table et son message brut ;
4. un incident de quasi-doublon, un débat sur la rétention, puis un brief décisionnel.

Chaque étape se referme sur les cinq mêmes éléments : une action, une confiance, une preuve, une incertitude et une limite.

Vote initial – la décision est-elle justifiée ?

A décision justifiée B plausible mais non auditable

C maintenir l'inspection par prudence D annuler la décision

À noter individuellement

Votre confiance de 0 à 100 %, le seul fait dont vous disposez réellement, la preuve que vous exigeriez en priorité, et le coût d'une erreur dans un sens ou dans l'autre.

Une chaîne de justification comporte quatre niveaux

Message brut → ligne structurée
→ indicateur → action

Chaque flèche de cette chaîne doit pouvoir nommer sa règle de transformation, la version de cette règle au moment des faits, et une trace permettant de la rejouer plus tard.

La prudence opérationnelle ne remplace pas la preuve

Déclencher une inspection peut rester une décision rationnelle si le coût d'un faux négatif – ignorer un vrai risque – est élevé. Cela ne démontre en rien que la mesure de 35,4 °C ayant motivé cette décision est exacte.

Deux questions distinctes, à ne jamais confondre

L'action décidée était-elle prudente compte tenu du risque ? La justification apportée par les données est-elle, elle, réellement auditée ?

Le brut est conservé au plus près de la réception

- ▶ Il possède déjà une structure : une enveloppe de réception et un payload publié par l'émetteur.
- ▶ Il peut contenir des métadonnées propres à la collecte elle-même, comme l'instant de réception.
- ▶ Il n'est pour autant ni pur ni automatiquement vrai : un message brut peut très bien contenir une erreur du capteur.
- ▶ Il permet, et c'est l'essentiel, de reparser le lot entier avec une règle nouvelle si celle utilisée aujourd'hui se révèle insuffisante.

Structurer, c'est projeter sous un contrat

Ce que l'on gagne

- ▶ des colonnes stables, aux noms et types fixés à l'avance ;
- ▶ la possibilité de calculer directement des indicateurs ;
- ▶ des contrôles de qualité systématiques et automatisables.

Ce que l'on risque

- ▶ des champs du message d'origine simplement omis dans la table ;
- ▶ des types forcés qui masquent une valeur douteuse ;
- ▶ une part du contexte original perdue en cours de route.

Question décisionnelle

Si ce champ disparaissait de la table, quelle contestation deviendrait alors impossible à formuler ?

La provenance relie entités, activités et responsabilités

La provenance décrit ce qui a servi de source, ce qui a été produit à partir d'elle, et par quelle activité précise cette production a eu lieu.

Version minimale retenue pour cette séquence

Pour chaque ligne structurée : le fichier source exact, son numéro de ligne, une empreinte cryptographique de cette ligne, la version du schéma appliqué, ainsi que le code et la commande exécutés pour produire cette ligne.

Source : W3C, PROV-Overview et PROV-DM, 2013.

Auditabilité ne signifie pas exactitude

Ce que l'auditabilité permet	Ce qu'elle ne garantit pas pour autant
retrouver précisément la source d'une ligne	la vérité physique de la mesure elle-même
rejouer intégralement une transformation	la calibration du capteur d'origine
détecter tout changement dans les octets	l'identité réelle de l'auteur du message
contester ouvertement une règle appliquée	l'exhaustivité du lot de messages

Une chaîne parfaitement auditable peut donc, malgré tout, reposer sur une mesure fausse : l'audit porte sur le traitement, pas sur le monde physique.

Enveloppe, payload et pipeline ont des autorités différentes

Niveau	Exemples de champs	Question à laquelle il répond
enveloppe	topic, received_at	qu'a observé la chaîne de collecte, et quand ?
payload	measured_at, value	que déclare l'émetteur du message lui-même ?
pipeline	source_line, raw_sha256	comment retrouver et vérifier le message brut ?

Ces trois niveaux ne se valident jamais l'un l'autre : le pipeline peut être parfaitement fiable tout en transportant fidèlement une valeur de capteur erronée.

Auto-vérification avant de continuer

Le brut est-il « sans structure » par opposition au CSV « structuré » ?

Non : le brut possède déjà une structure (enveloppe et payload JSON). Structurer signifie ici projeter cette structure existante sous un schéma de colonnes stable, pas lui donner une structure qu'il n'aurait pas.

Une chaîne parfaitement auditable garantit-elle une mesure physiquement exacte ?

Non : l'auditabilité permet de retrouver la source et de rejouer la transformation, mais elle ne calibre aucun capteur et ne certifie aucune vérité de terrain.

Que répond chacun des trois niveaux – enveloppe, payload, pipeline ?

L'enveloppe décrit ce que la collecte a observé ; le payload rapporte ce que l'émetteur déclare ; le pipeline documente comment retrouver et vérifier le message d'origine.

TP 1 – Prédire avant de parser

1. Combien d'enveloppes attendez-vous en sortie, à partir des 15 messages du fichier brut ?
2. Quels champs proviennent respectivement de l'enveloppe, du payload et du pipeline lui-même ?
3. Que doit-il se passer si une ligne du fichier se révèle incomplète pendant le parsing ?
4. Quelle phrase seriez-vous déjà capable de défendre avant même d'avoir exécuté la commande ?

TP 1 – Produire le CSV structuré

```
export PYTHONPATH=src
python3 -m iot_decision.traceability_cli parse \
  data/samples/batch001_messages.jsonl \
  data/processed/batch001_structured.csv
```

Résultat attendu

15 lignes structurées, réparties sur 17 colonnes ; aucune anomalie isolée sur ce lot précis.

TP 1 – Contrôler avant d'interpréter

```
head -n 3 data/processed/batch001_structured.csv  
tail -n +2 data/processed/batch001_structured.csv | wc -l
```

Résultat attendu : une ligne d'en-tête complète avec 17 noms de colonnes, suivie de 15 lignes de données – ni plus, ni moins.

Le mode strict rend l'échec visible

Un payload incomplet doit arrêter immédiatement le traitement du lot entier. Un futur mode tolérant devra, lui, isoler précisément l'erreur et produire une trace explicite de ce qui a été rejeté.

Exemple concret testé en séance

Un message dont le payload est vide (`{}`) provoque un arrêt immédiat avec la liste exacte des champs manquants – jamais un remplacement silencieux par une valeur vide.

Interdit dans ce pipeline

Remplacer silencieusement un champ obligatoire absent par une chaîne vide : cela transformerait une donnée manquante en fausse donnée présente.

Point d'étape – formulation exacte

“15 enveloppes conformes au contrat s03-v1 ont été structurées.”

Ce que cette phrase ne démontre pas

Ni l'exhaustivité du lot d'origine, ni l'exactitude physique des valeurs, ni l'authenticité de leur émetteur, ni la qualité des horloges qui ont produit les horodatages.

TP 2 – Défendre le schéma champ par champ

Champ	Origine	Contrôle	Risque si ce champ était supprimé
topic	enveloppe	non vide	le périmètre de publication devient impossible à vérifier
unit	payload	vocabulaire connu	deux valeurs pourraient être comparées sans partager d'unité
source_line	pipeline	entier ≥ 1	le retour au message brut devient ambigu
schema_version	pipeline	valeur connue	la règle de parsing appliquée devient inconnue

Pourquoi garder deux horloges ?

`measured_at` est déclaré directement par l'émetteur du message.

`received_at` est observé indépendamment par la chaîne de collecte.

Utilité concrète

Comparer ces deux horodatages permet d'étudier un délai apparent entre mesure et réception ; cela ne valide en rien l'exactitude de l'une ou l'autre horloge physique.

Source : IETF, RFC 3339, 2002.

Pourquoi `message_id` et `source_line` ?

- ▶ `message_id` est une identité déclarée par l'émetteur lui-même, donc jamais vérifiée de façon indépendante.
- ▶ `source_line` est une adresse physique dans le fichier, attribuée par notre propre pipeline, pas par l'émetteur.
- ▶ Aucun des deux, séparément, ne prouve l'identité réelle de l'auteur du message.
- ▶ Leur désaccord – par exemple deux lignes différentes partageant le même `message_id` – doit systématiquement déclencher une enquête.

Auto-vérification avant de continuer

Pourquoi conserver `topic` si la colonne `zone` existe déjà ?

Parce que `zone` seule ne permet plus de vérifier a posteriori la branche de topics et le périmètre exacts qui ont réellement été interrogés lors de l'extraction.

`measured_at` et `received_at` sont-elles interchangeables ?

Non : l'une est déclarée par l'émetteur, l'autre observée par la collecte. Comparer les deux renseigne sur un délai apparent, sans jamais valider la justesse d'aucune des deux horloges.

Que faut-il faire si un `payload` est incomplet, en mode strict ?

Arrêter immédiatement le traitement du lot avec la liste exacte des champs manquants, jamais remplacer silencieusement le champ absent par une valeur vide.

Pause – 10 minutes

Au retour : que prouve vraiment une empreinte cryptographique – et
que ne prouve-t-elle pas ?

Une empreinte détecte une différence avec une référence

`raw_sha256` est calculée sur le contenu exact de la ligne brute, sans le saut de ligne final.

Ce qu'elle prouve

Que la ligne comparée produit – ou non – exactement la même empreinte que la référence enregistrée.

Ce qu'elle ne prouve jamais

Ni l'auteur du message, ni son antériorité réelle, ni son authenticité, ni la vérité physique de la mesure qu'il transporte.

Canoniser et préserver sont deux stratégies

- ▶ Préserver consiste à calculer l'empreinte des octets reçus tels quels : elle devient sensible au moindre changement, y compris purement cosmétique.
- ▶ Canoniser consiste à construire une représentation stable du contenu avant tout calcul cryptographique, pour ignorer les variations cosmétiques.
- ▶ Cette séquence choisit de préserver la ligne brute telle quelle, et documente cette convention explicitement dans le schéma.
- ▶ Une conséquence directe : re-sérialiser le même JSON peut changer les octets produits sans changer aucune des valeurs qu'il contient.

Source : IETF, RFC 8785, JSON Canonicalization Scheme, 2020.

Classement – démontré, plausible ou non démontré ?

Pour chacune des affirmations suivantes, décidez si elle est démontrée par nos artefacts, seulement plausible, ou non démontrée du tout :

1. le champ `message_id` est bien présent dans la ligne ;
2. le capteur nommé dans le message est réellement son auteur authentique ;
3. le contenu brut et la ligne CSV de référence se correspondent exactement ;
4. l'horloge interne du capteur est réglée avec exactitude ;
5. la transformation appliquée peut être rejouée à l'identique par un tiers.

Auto-vérification avant de continuer

Une empreinte identique entre le brut et le CSV prouve-t-elle l'authenticité de l'émetteur ?

Non : elle prouve seulement que les octets comparés correspondent à la référence enregistrée – rien sur qui a produit cette référence à l'origine, ni sur son antériorité.

« Le message_id est présent » et « le capteur nommé est son auteur authentique » sont-elles au même niveau de preuve ?

Non : la première est directement démontrée par simple lecture du champ ; la seconde reste non démontrée, car rien dans le pipeline ne vérifie l'identité réelle de l'émetteur.

Préserver et canoniser produisent-ils la même empreinte pour un JSON re-sérialisé ?

Pas nécessairement : préserver capture les octets exacts reçus, donc sensibles à une re-sérialisation ; canoniser vise justement à neutraliser ce genre de variation cosmétique.

TP 3 – Vérifier tout le lot

```
python3 -m iot_decision.traceability_cli verify \  
  data/samples/batch001_messages.jsonl \  
  data/processed/batch001_structured.csv
```

Résultat attendu

traçabilité vérifiée : chaque ligne du CSV reproduit exactement l'empreinte de sa ligne brute déclarée.

TP 3 – Rejouer deux liens manuellement

Choisissez par exemple les lignes CSV 3 et 14, puis pour chacune :

1. relevez son fichier source, son numéro de ligne, son `message_id` et son empreinte `raw_sha256` ;
2. affichez la ligne correspondante du fichier JSONL brut avec `sed -n` ;
3. comparez chaque champ entre la ligne structurée et la ligne brute affichée ;
4. écrivez explicitement une conclusion autorisée par cette comparaison, et une conclusion qu'elle interdit.

Le saut de ligne fait partie de la convention

Calculer `sha256sum` sur la sortie de `sed` inclut le saut de ligne final de cette ligne.
Notre pipeline, lui, calcule l'empreinte sans ce saut de ligne.

Réflexe à avoir

Une empreinte qui semble diverger doit d'abord faire examiner la convention de calcul utilisée, avant même de suspecter l'entrée elle-même.

Test réversible – altérez seulement une copie

Copiez le fichier brut vers /tmp, modifiez la valeur 33.8 en 33.9 dans cette copie, puis relancez la vérification sur cette copie altérée.

Résultat attendu

Une empreinte différente est signalée précisément pour `source_line=1` – et seulement pour celle-là. Le fichier brut de référence, lui, reste parfaitement intact.

Incident – deux messages semblent identiques

Deux messages du lot d'incident partagent exactement même site, même zone, même capteur, même instant mesuré, même valeur, même unité et même numéro de séquence.

Question décisionnelle

S'agit-il de messages réellement identiques, de doublons certains, de simples candidats doublons à signaler, ou d'événements distincts ?

Comparer tous les champs change le diagnostic

Si l'on compare, au-delà des champs métier déjà identiques ci-dessus, les métadonnées de collecte et de pipeline, les deux messages divergent nettement :

Champ	Message A	Message B
message_id	...-0002	...-replay-a
received_at	10 :05 :00	10 :05 :02
retained	true	false
topic (suffixe)	/0002	/replay-a
raw_sha256	différent	différent

Même valeur mesurée ne veut donc pas dire même message : ces deux enregistrements restent deux événements de collecte distincts, même s'ils décrivent apparemment la même mesure.

Un candidat doublon n'autorise pas la suppression

Une règle de déduplication défendable doit nommer explicitement cinq éléments :

- ▶ la clé de comparaison utilisée et la fenêtre de temps considérée ;
- ▶ l'autorité habilitée à approuver une telle règle ;
- ▶ le risque de faux positif, c'est-à-dire de fusionner à tort deux événements réellement distincts ;
- ▶ le sort réservé à la ligne écartée, et la trace qui en est conservée ;
- ▶ la procédure permettant de rejouer la décision si la règle se révèle plus tard incorrecte.

Auto-vérification avant de continuer

Deux messages partageant la même valeur mesurée sont-ils nécessairement le même événement ?

Non : ce cas d'incident montre deux messages avec une valeur, une zone et un instant identiques, mais des identifiants, horodatages de réception, indicateurs retained et empreintes tous différents – ce sont deux événements de collecte distincts.

Un pipeline peut-il supprimer automatiquement un candidat doublon détecté ?

Non dans cette séquence : la fonction de détection se contente de regrouper les candidats sans jamais en supprimer aucun ; toute suppression exigerait une règle explicite, autorisée et tracée.

Mini-débat – supprimer le brut après le CSV ?

Arguments pour conserver

- ▶ permettre un reparsing si la règle change ;
- ▶ garder la preuve opposable en cas de contestation ;
- ▶ permettre une investigation future sur un incident.

Arguments pour supprimer

- ▶ minimiser les données réellement conservées ;
- ▶ réduire l'exposition en confidentialité ;
- ▶ réduire le coût de stockage et la surface d'attaque.

Produisez une politique de compromis explicite : durée de rétention, autorité responsable, contrôle d'accès, mécanisme d'intégrité et condition de destruction.

Le brief final tient en sept éléments

1. une action et un périmètre précisément délimité ;
2. une confiance technique, portant sur la chaîne de traitement ;
3. une confiance opérationnelle, portant sur la décision elle-même ;
4. deux preuves chiffrées et retrouvables dans les fichiers produits ;
5. deux incertitudes qui pourraient changer la décision ;
6. une limite assumée et explicitement formulée ;
7. une vérification prioritaire à mener avant toute action irréversible.

150 mots maximum, pas un de plus.

Une réponse recevable sépare les deux confiances

Structure attendue

La correspondance entre le brut et la table est vérifiée de bout en bout : confiance technique élevée. La décision opérationnelle, elle, ne peut recevoir mieux qu'une confiance moyenne, car ni le capteur, ni son horloge, ni l'exhaustivité du lot ne sont validés par cette seule chaîne.

Exit ticket – trois phrases sans écran

1. Je peux justifier... parce que...
2. Je ne peux pas affirmer...
3. Je conserverais... pendant... sous l'autorité de...

Réponse à la question directrice

Question décisionnelle

Quelles informations faut-il absolument conserver pour pouvoir justifier une décision plus tard ?

Le message brut lui-même, son adresse exacte et son empreinte, ses identifiants et leur contexte, ses deux horloges (mesure et réception), son unité, la version du schéma appliqué, le code et la commande exécutés, ainsi qu'un journal des erreurs rencontrées. Sur ce lot précis : 15 messages bruts produisent 15 lignes structurées sur 17 colonnes, et la vérification confirme que chaque ligne correspond exactement à son message d'origine. **Niveau de confiance** : élevée pour la correspondance entre le brut et la table ; moyenne au mieux pour la décision opérationnelle elle-même, faute de validation du capteur, de son horloge et de l'exhaustivité du lot.

Références

- ▶ W3C, *PROV-Overview* et *PROV-DM*, 2013. <https://www.w3.org/TR/prov-overview/>
- ▶ IETF, *RFC 3339 – Date and Time on the Internet*, 2002.
<https://www.rfc-editor.org/rfc/rfc3339>
- ▶ IETF, *RFC 8785 – JSON Canonicalization Scheme*, 2020.
<https://www.rfc-editor.org/rfc/rfc8785>
- ▶ Python Software Foundation, *json – JSON encoder and decoder*.
<https://docs.python.org/3/library/json.html>